

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-10 Créditos: 3	
---	--	---

Taller 3: Investigación Criminalística con Lógica

Grupo 8

Por: Karen Fuentes (202122467), Sofia Sarasty (202511871), Juan Pablo Camacho Peña (202110977)

Distribución del trabajo

Integrante	Sección	Tareas
Karen Fuentes	Punto 1 y Parte Punto 2	Model Checking completo + Funciones iniciales de CNF
Sofia Sarasty	Punto 2	Funciones restantes de CNF (Distribute, Flatten)
Juan Pablo Camacho	Punto 3 y Punto 4	Casos Criminales (Predicados), Análisis y Reflexión

Tabla de Contenidos

Tabla de Contenidos.....	2
1. Introducción y Descripción del Proyecto.....	4
2. Punto 1: Model Checking Proposicional.....	4
2.1 Fundamentos Teóricos.....	4
2.2 Implementación.....	4
2.2.1 Generación de Modelos: <code>get_all_models</code>	4
2.2.2 Satisfacibilidad: <code>check_satisfiable</code>	5
2.2.3 Validez (Tautología): <code>check_valid</code>	5
2.2.4 Implicación Lógica: <code>check_entailment</code>	5
2.2.5 Tabla de Verdad: <code>truth_table</code>	5
2.3 Resultados y Verificación.....	5
3. Punto 2: Transformación a Forma Normal Conjuntiva (CNF).....	6
3.1 Fundamentos Teóricos.....	6
3.2 Pipeline de Transformación.....	6
3.3 Implementaciones Detalladas.....	6
3.3.1 Eliminación de Bicondicionales (<code>eliminate_iff</code>).....	6
3.3.2 Eliminación de Implicaciones (<code>eliminate_implication</code>).....	6
3.3.3 Propagación de Negaciones hacia Adentro (<code>push_negation_inward</code>).....	6
3.3.4 Distribución de OR sobre AND (<code>distribute_or_over_and</code>).....	7
3.3.5 Aplanamiento (<code>flatten</code>).....	7
3.4 Integración con Resolución.....	7
4. Punto 3: Lógica de Predicados — Casos Criminales.....	8
4.1 Fundamentos Teóricos.....	8

4.2 Estructura de la Base de Conocimiento.....	8
4.3 Los Cinco Casos Criminales.....	8
4.3.1 El Veneno de Villa Espinas.....	8
4.3.2 El Robo en el Expreso del Sur.....	8
4.3.3 El Sabotaje en Laboratorio Pharmax.....	9
4.3.4 La Herencia Maldita de Hacienda El Rosal.....	9
4.3.5 La Red del Puerto de las Sombras.....	9
4.4 Técnicas de Inferencia Implementadas.....	10
4.4.1 Forward Chaining.....	10
4.4.2 Backward Chaining.....	10
4.5 Resumen de Resultados.....	10
5. Distribución del Trabajo.....	12
6. Conclusiones.....	12
7. Referencias.....	13

1. Introducción y Descripción del Proyecto

El presente informe documenta el diseño, implementación y análisis del Taller 3 de la asignatura ISIS-1611 Inteligencia Artificial de la Universidad de los Andes. El proyecto consiste en la construcción de un motor de inferencia lógica aplicado a la resolución de crímenes ficticios, combinando dos paradigmas fundamentales de la lógica formal: la lógica proposicional y la lógica de predicados.

El sistema integra múltiples algoritmos clásicos de inteligencia artificial: verificación de modelos (model checking), transformación a Forma Normal Conjuntiva (CNF), resolución proposicional, y encadenamiento hacia adelante y hacia atrás sobre cláusulas de Horn. Estos algoritmos fueron implementados en Python y orquestados mediante una interfaz de terminal interactiva (TUI) construida con la librería Textual.

El proyecto se divide en tres grandes componentes: (1) Model Checking proposicional, que permite razonar sobre todas las posibles asignaciones de verdad; (2) Transformaciones a CNF, que convierten fórmulas lógicas a una forma estandarizada para aplicar resolución; y (3) Lógica de Predicados, donde se construyen bases de conocimiento para cinco casos criminales distintos y se aplican motores de inferencia para identificar culpables.

2. Punto 1: Model Checking Proposicional

2.1 Fundamentos Teóricos

El model checking proposicional es una técnica de verificación formal que consiste en explorar exhaustivamente todos los posibles mundos (modelos) en los que pueden ser verdaderas o falsas las proposiciones atómicas de un sistema lógico. Para n átomos, existen exactamente 2^n modelos posibles, cada uno representando una asignación completa de valores de verdad.

Esta técnica permite responder preguntas fundamentales sobre fórmulas lógicas: si una fórmula es satisfacible (existe al menos un modelo que la hace verdadera), si es una tautología (es verdadera en todos los modelos posibles), y si una base de conocimiento implica lógicamente una consulta (en todo modelo que satisface la KB, la consulta también es verdadera).

2.2 Implementación

2.2.1 Generación de Modelos: `get_all_models`

La función `get_all_models` genera las 2^n asignaciones de verdad para n átomos. La estrategia implementada utiliza representación binaria: cada entero i en el rango $[0, 2^n - 1]$ codifica un modelo, donde el bit j -ésimo (del más significativo al menos significativo) determina el valor del j -ésimo átomo en orden lexicográfico.

```
def get_all_models(atoms: set[str]) -> list[dict[str, bool]]:

    lista_atomos = sorted(atoms) # orden determinístico

    n = len(lista_atomos)

    modelos = []

    for i in range(2 ** n):

        modelo = {}

        for j, atom in enumerate(lista_atomos):

            modelo[atom] = bool((i >> (n - 1 - j)) & 1)

        modelos.append(modelo)

    return modelos
```

La expresión $(i \gg (n-1-j)) \& 1$ aísla el bit j del número i . Al ordenar los átomos lexicográficamente y usar el bit más significativo para el primer átomo, se garantiza un orden consistente y reproducible (TT, TF, FT, FF para dos átomos).

2.2.2 Satisfacibilidad: `check_satisfiable`

Una fórmula es satisfacible si existe al menos un modelo en el que evalúa a verdadero. La implementación itera sobre todos los modelos generados por `get_all_models` y retorna el primer modelo satisfaciente encontrado, o (False, None) si ninguno satisface la fórmula.

2.2.3 Validez (Tautología): `check_valid`

Una fórmula es válida si es verdadera en la totalidad de los modelos posibles. La implementación verifica que no exista ningún contraejemplo: recorre todos los modelos y retorna False en cuanto encuentra uno donde la fórmula es falsa.

2.2.4 Implicación Lógica: `check_entailment`

La implicación lógica $KB \models q$ se verifica aplicando el teorema de refutación: KB implica q si y solo si $KB \wedge \neg q$ es insatisfacible. Para ello se construye la conjunción de todas las fórmulas de la KB con la negación de la consulta, y se verifica que esta fórmula compuesta sea insatisfacible.

```
def check_entailment(kb, query):

    formula = And(*kb, Not(query)) if kb else Not(query)

    satisfiable, _ = check_satisfiable(formula)

    return not satisfiable
```

2.2.5 Tabla de Verdad: `truth_table`

La función `truth_table` combina `get_all_models` con `evaluate` para producir la lista completa de pares (modelo, resultado) que conforman la tabla de verdad de una fórmula, con todos sus 2^n filas.

2.3 Resultados y Verificación

La batería de pruebas definida en `tests/test_model_checking.py` valida el correcto funcionamiento de todas las funciones. Los tests incluyen desde casos básicos (contradicciones, tautologías simples) hasta casos avanzados como el razonamiento criminal: dado que el jardín está mojado y la lluvia implica jardín mojado, se puede inferir que llovió.

Ejemplo de Razonamiento Criminal con Model Checking

KB = {"si no llovía, entonces el jardín no estaba mojado", "el jardín está mojado"} Consulta: ¿Llovía?
Resultado: `check_entailment(KB, llovía) → True` (lógicamente se desprende que llovió)

3. Punto 2: Transformación a Forma Normal Conjuntiva (CNF)

3.1 Fundamentos Teóricos

La Forma Normal Conjuntiva (CNF) es una representación canónica de fórmulas lógicas en la que toda fórmula se expresa como una conjunción (AND) de cláusulas, donde cada cláusula es una disyunción (OR) de literales. Un literal es un átomo o su negación directa. Toda fórmula proposicional puede convertirse a CNF manteniendo equivalencia lógica.

La CNF es fundamental para el algoritmo de resolución proposicional: al operar con cláusulas uniformes, la resolución puede derivar nuevas cláusulas por eliminación de literales complementarios, y probar implicaciones lógicas al intentar derivar la cláusula vacía (contradicción).

3.2 Pipeline de Transformación

La conversión a CNF se realiza mediante un pipeline secuencial de seis transformaciones, cada una aplicada recursivamente sobre el árbol sintáctico abstracto (AST) de la fórmula. El orden es crítico, pues cada paso prepara la fórmula para el siguiente.

Paso	Función	Transformación	Implementador
1	eliminate_iff	$A \leftrightarrow B \rightarrow (A \rightarrow B) \wedge (B \rightarrow A)$	Equipo (implementado)
2	eliminate_implication	$A \rightarrow B \rightarrow \neg A \vee B$	Equipo (implementado)
3	push_negation_inward	$\neg(A \wedge B) \rightarrow \neg A \vee \neg B$ / $\neg(A \vee B) \rightarrow \neg A \wedge \neg B$	Equipo (implementado)
4	eliminate_double_negation	$\neg\neg A \rightarrow A$	Suministrado (dado)
5	distribute_or_over_and	$A \vee (B \wedge C) \rightarrow (A \vee B) \wedge (A \vee C)$	Equipo (implementado)
6	flatten	$\text{And}(\text{And}(a,b),c) \rightarrow \text{And}(a,b,c)$	Equipo (implementado)

3.3 Implementaciones Detalladas

3.3.1 Eliminación de Bicondicionales (eliminate_iff)

El bicondicional $A \leftrightarrow B$ se reemplaza por la conjunción de las dos implicaciones equivalentes: $(A \rightarrow B) \wedge (B \rightarrow A)$. La función es puramente recursiva: desciende por el AST y sólo transforma los nodos de tipo `Iff`, pasando los demás sin modificación.

3.3.2 Eliminación de Implicaciones (eliminate_implication)

La implicación $A \rightarrow B$ se reemplaza por su equivalente lógica $\neg A \vee B$. Esto transforma el AST en una fórmula que solo contiene los conectivos NOT, AND y OR, preparándolo para los pasos siguientes.

3.3.3 Propagación de Negaciones hacia Adentro (push_negation_inward)

Esta función aplica las leyes de De Morgan para mover las negaciones hacia los átomos. Al encontrar un `Not` sobre un `And`, lo convierte en `Or` de negaciones; al encontrar `Not` sobre un `Or`, lo convierte en `And` de negaciones. Al encontrar una doble negación `Not(Not(x))`, la elimina directamente. El proceso es recursivo hasta que todas las negaciones están directamente sobre los átomos.

```
if isinstance(inner, And):
    return Or(*[push_negation_inward(Not(c)) for c in inner.conjuncts])

if isinstance(inner, Or):
    return And(*[push_negation_inward(Not(d)) for d in inner.disjuncts])
```

3.3.4 Distribución de OR sobre AND (distribute_or_over_and)

Esta es la transformación más compleja del pipeline. Al encontrar un nodo `Or` cuyos hijos contienen algún nodo `And`, aplica la regla $A \vee (B \wedge C) \rightarrow (A \vee B) \wedge (A \vee C)$, distribuyendo el `Or` sobre cada conjunción. La función es recursiva y puede necesitar múltiples pasadas para manejar distribuciones anidadas. Se detecta el primer hijo de tipo `And`, se extraen los demás disjuntos, y se genera la nueva conjunción de disyunciones.

3.3.5 Aplanamiento (flatten)

Elimina el anidamiento innecesario de conjunciones y disyunciones: `And(And(a,b),c)` pasa a ser `And(a,b,c)`. Esto produce una CNF en su forma más limpia, con cláusulas que son directamente `frozensets` de literales para el módulo de resolución.

3.4 Integración con Resolución

El módulo `resolution.py` (suministrado) convierte la CNF en conjuntos de cláusulas y aplica el algoritmo de resolución por refutación: añade la negación de la consulta a la KB, y busca derivar la cláusula vacía. Si lo logra, la implicación queda probada. Esto complementa el `model checking` como segundo método de prueba de implicaciones lógicas.

4. Punto 3: Lógica de Predicados — Casos Criminales

4.1 Fundamentos Teóricos

La lógica de predicados de primer orden extiende la lógica proposicional al permitir razonar sobre objetos individuales y sus relaciones mediante predicados y variables. En este proyecto se utiliza una forma restringida conocida como cláusulas de Horn: reglas con exactamente una conclusión positiva en la cabeza y cero o más condiciones en el cuerpo.

Las cláusulas de Horn permiten dos estrategias de inferencia eficientes: el encadenamiento hacia adelante (forward chaining), que parte de los hechos base y deriva todos los hechos posibles hasta alcanzar un punto fijo; y el encadenamiento hacia atrás (backward chaining), que parte de una meta y trabaja recursivamente hacia atrás buscando hechos que la sustenten.

La unificación es el mecanismo central que permite a ambos motores relacionar predicados con variables con hechos concretos, produciendo sustituciones que asignan valores a las variables.

4.2 Estructura de la Base de Conocimiento

Cada caso criminal se modela como una KnowledgeBase compuesta de dos tipos de elementos: hechos (Facts), que son predicados ground sin variables que se consideran siempre verdaderos; y reglas (Rules), que son cláusulas de Horn de la forma $\text{head} \text{ :- body1, body2, ..., donde la cabeza se puede probar cuando todos los predicados del cuerpo son verdaderos.}$

La convención del proyecto establece que las variables comienzan con \$ (ej: \$X, \$Y), mientras que las constantes no llevan este prefijo (ej: reynaldo, frasco_arsenico). Esta distinción es fundamental para el algoritmo de unificación.

4.3 Los Cinco Casos Criminales

4.3.1 El Veneno de Villa Espinas

Don Evaristo fue hallado muerto con sedante disuelto en su medicación nocturna. Se modelan los siguientes hechos y reglas de deducción:

- Hechos base: huellas de Reynaldo en el frasco de arsénico, Pablo y Bernardo estaban lejos de la escena, acusación de Pablo contra Reynaldo, coartadas cruzadas entre Reynaldo y Margot.
- Reglas clave: quien tiene huellas en el arma del crimen tiene evidencia directa; quien tiene evidencia directa y no tiene coartada verificada es culpable; quien da coartada a un culpable lo está encubriendo; dos personas que se dan coartada

mutuamente tienen una coartada cruzada.

- Conclusión: Reynaldo es culpable. Margot lo encubre. Pablo y Bernardo están descartados (coartada verificada de cámara). Existe coartada cruzada entre Margot y Reynaldo.

4.3.2 El Robo en el Expreso del Sur

El collar de esmeraldas de la Marquesa desapareció del vagón privado. Elena fue vista en el vagón con huellas en el estuche. Don Rodrigo tiene coartada de cámara en el vagón de equipaje. La Marquesa acusa a Elena como testigo imparcial. Víctor y Elena se cubren mutuamente, formando una alianza de coartadas.

- Hechos base: Elena en la escena con huellas, Don Rodrigo grabado en cámara lejos de la escena, la Marquesa como víctima (testigo imparcial), acusaciones y declaraciones cruzadas de coartada.
- Reglas: cámara verifica coartada y descarta al sospechoso; la víctima es testigo imparcial; acusación de testigo imparcial es creíble; quien está en la escena con acusación creíble es culpable; coartadas mutuas forman alianza.
- Conclusión: Elena es culpable. Don Rodrigo está descartado. Víctor defiende al culpable. Existe alianza de coartadas entre Víctor y Elena.

4.3.3 El Sabotaje en Laboratorio Pharmax

Los cultivos del Proyecto Ámbar fueron destruidos. El Técnico Ríos recibió pagos de Syntek Corp. (empresa rival) y entró a la sala de cultivos. La Dra. Santos y el Director Vega tienen coartadas verificadas de viaje internacional y conferencia, respectivamente.

- Hechos base: Dra. Santos y Director Vega con documentación oficial de ausencia; Técnico Ríos despedido con pagos de Syntek; Asistente Mora amenazado con despido; acceso registrado de ambos a la sala; Mora acusa a Ríos.
- Reglas: documentación oficial es coartada verificada; coartada verificada descarta sospechoso; pagos de empresa beneficiada crean conflicto de intereses (motivo económico); acceso registrado al lugar constituye presencia en el momento; sin coartada + motivo económico + presencia = culpable; quien también estuvo en el lugar hace una denuncia informada.
- Conclusión: Técnico Ríos es culpable (tiene motivo económico, acceso registrado y sin coartada). La denuncia de Mora es informada. Dra. Santos y Director Vega están descartados.
- ForallGoal verificado: todo culpable tuvo acceso en el momento del sabotaje.

4.3.4 La Herencia Maldita de Hacienda El Rosal

Don Evaristo fue hallado muerto con sedante en su medicación. Había anunciado cambiar su testamento al día siguiente. El Sobrino Esteban tiene huellas en el vaso adulterado y acusa a la

Secretaria Luna. El Abogado Restrepo acusa a Esteban. La Secretaria Luna declara que Esteban estuvo con ella (coartada).

- Hechos base: Enfermera Campos con coartada verificada por cámara; Restrepo y Esteban heredan y perderían con el cambio; Luna gana con el nuevo testamento; huellas de Esteban en el vaso adulterado; acusaciones cruzadas.
- Reglas: coartada verificada descarta; heredar y perder con el cambio es motivo doble; huellas en el objeto del crimen son evidencia física; motivo doble + sin coartada + evidencia física = culpable; el culpable que acusa a otro hace un desvío sospechoso; quien da coartada al culpable encubre; acusación corroborada si acusador tiene motivo doble y acusado tiene evidencia física.
- Conclusión: Sobrino Esteban es culpable. Tiene motivo doble, sin coartada y evidencia física. Su acusación a Luna es un desvío sospechoso. La acusación de Restrepo contra Esteban está corroborada. Enfermera Campos está descartada.

4.3.5 La Red del Puerto de las Sombras

Mercancía ilegal fue encontrada en el Puerto Industrial. El Oficial Duarte firma manifiestos fraudulentos. El Marinero Pinto fue visto introduciendo mercancía ilegal. Ambos pertenecen al mismo cartel. El Capitán Herrera y el Inspector Nova tienen registros oficiales de ausencia verificados.

- Hechos base: Herrera con registro digital de salida verificado; Nova con documentación oficial fuera del puerto; Duarte firma manifiestos fraudulentos sin coartada; Pinto con acceso irrestricto a bodega y visto introduciendo mercancía, sin coartada; ambos en el mismo cartel; informante reportó a ambos; Herrera acusa a Duarte.
- Reglas: registro oficial fuera del puerto descarta; manifiestos fraudulentos = fraude documental; visto introduciendo mercancía = introduce contrabando; fraude documental sin coartada = culpable; contrabando sin coartada = culpable; mismo cartel = comparten red; dos culpables que comparten red = operación conjunta; testimonio de descartado es confiable; red activa si al menos un miembro es culpable.
- Conclusión: Duarte y Pinto son culpables. Operan de forma conjunta. El testimonio de Herrera contra Duarte es confiable. La red cartelaria está activa.
- ExistsGoal y ForallGoal verificados: existe una red activa; todo reportado por el informante es culpable.

Punto 3b: Preguntas de Análisis Cualitativo (15%)

A continuación se responden de forma razonada las cinco preguntas de análisis planteadas por el taller, tomando como referencia directa la experiencia de modelado de los cinco casos criminales implementados en la carpeta crimes/ del proyecto.

Pregunta 1: Evidencia directa vs. indirecta en predicados y reglas

Pregunta

¿Cómo se modela la diferencia entre evidencia directa (por ejemplo, huellas en el arma) e indirecta (por ejemplo, un testimonio) usando predicados y reglas?

La diferencia entre evidencia directa e indirecta se refleja naturalmente en la longitud y la estructura de la cadena de inferencia necesaria para derivar un predicado como `evidencia_directa` o `acusacion_creible`.

Evidencia directa

La evidencia directa establece un vínculo material e inmediato entre el sospechoso y el crimen. Se modela con hechos atómicos que no requieren mediadores: la existencia física de una huella, de un registro de cámara o de un objeto hallado en posesión del sospechoso. La regla que la instancia es breve: necesita como máximo dos predicados en el cuerpo.

```
# Hecho: las huellas de Reynaldo están en el frasco de arsénico (objeto del crimen)

kb.add_fact(Predicate('huellas_en', (Term('reynaldo'), Term('frasco_arsenico'))))

kb.add_fact(Predicate('arma_del_crimen', (Term('frasco_arsenico'))))

# Regla: quien tiene huellas en el arma tiene evidencia directa

kb.add_rule(Rule(

    head=Predicate('evidencia_directa', (Term('$X'))),

    body=(

        Predicate('huellas_en', (Term('$X'), Term('$A'))),

        Predicate('arma_del_crimen', (Term('$A'))),
```

La cadena inferencial es corta: un único paso de unificación entre los hechos base y la regla produce `evidencia_directa(reynaldo)`. No se necesita evaluar la credibilidad de ninguna fuente ni pasar por predicados intermedios de confiabilidad.

Evidencia indirecta (testimonial)

La evidencia indirecta requiere una cadena inferencial más larga porque su credibilidad depende de la naturaleza de la fuente. Un testimonio solo constituye evidencia si quien lo emite es a su vez confiable, y la confiabilidad de un testigo se deriva de que este no sea sospechoso (es decir, que esté descartado como culpable).

```
# Hecho: Pablo acusa a Reynaldo

kb.add_fact(Predicate('acusa', (Term('pablo'), Term('reynaldo'))))


# Regla 1: quien está lejos de la escena está descartado

kb.add_rule(Rule(

    head=Predicate('descartado', (Term('$X'),)),

    body=(Predicate('lejos_de_escena', (Term('$X'),)),)

))


# Regla 2: el testimonio de un descartado es confiable

kb.add_rule(Rule(

    head=Predicate('testimonio_confiable', (Term('$T'), Term('$S'))),

    body=(

        Predicate('descartado', (Term('$T'),)),

        Predicate('acusa', (Term('$T'), Term('$S'))),

    )

))
```

En este caso, para probar `testimonio_confiable(pablo, reynaldo)` el motor debe primero establecer que pablo está descartado, lo que a su vez requiere probar `lejos_de_escena(pablo)`. La cadena es: `lejos_de_escena → descartado → testimonio_confiable → (eventualmente) acusacion_creible → culpable`. Esta profundidad de inferencia es la diferencia formal entre ambos tipos de evidencia.

Conclusión clave

La evidencia directa conecta al sospechoso con el crimen en un solo paso (`huellas → evidencia_directa`). La evidencia indirecta pasa por cadenas de predicados intermedios que validan la fuente antes de que su contenido pueda utilizarse como premisa para derivar culpabilidad.

Pregunta 2: Evidencia en contra y coartada verificada simultáneas

Pregunta

¿Qué sucede si un sospechoso tiene tanto evidencia en su contra como una coartada verificada?
¿El motor detecta esta contradicción? Explica con un ejemplo.

Comportamiento del motor bajo la suposición de mundo cerrado

El motor de forward/backward chaining implementado opera bajo la suposición de mundo cerrado (CWA) y no implementa lógica paraconsistente ni detección de contradicciones. Esto tiene una consecuencia directa y práctica: si un sospechoso tiene evidencia en su contra Y tiene coartada verificada, el sistema no genera ningún error, sino que simplemente no lo marca como culpable.

La razón es que la regla de culpabilidad tiene una conjunción en el cuerpo que incluye `sin_coartada($X)`. Si el sistema puede derivar `coartada_verificada($X)` y, a partir de esta, `descartado($X)`, pero no puede derivar `sin_coartada($X)` (porque esta es la negación de un hecho positivo), entonces la regla de culpabilidad nunca se dispara para ese individuo.

Ejemplo concreto: Técnico Ríos con coartada hipotética

```

# SITUACIÓN HIPOTÉTICA: Técnico Ríos con coartada Y evidencia en su contra

# Evidencia en contra (ya modelada en el caso real):

kb.add_fact(Predicate('recibio_pagos', (tec_rios, syntek_corp)))

kb.add_fact(Predicate('empresa_beneficiada', (syntek_corp,)))

kb.add_fact(Predicate('acceso_registrado', (tec_rios, sala_cultivos)))

kb.add_fact(Predicate('lugar_saboteado', (sala_cultivos,)))

# HIPOTÉTICA: coartada verificada añadida artificialmente

kb.add_fact(Predicate('documentacion_oficial', (tec_rios,)))

# Regla: documentación oficial → coartada_verificada → descartado

# => El motor deriva: descartado(tec_rios)

# Regla culpable requiere sin_coartada($X):

# culpable($X) :- motivo_economico($X), acceso_en_momento($X), sin_coartada($X)

# Como no hay hechos que permitan derivar sin_coartada(tec_rios),

# la regla NO se dispara => tec_rios NO es marcado culpable.

# RESULTADO:

# backward_chain(kb, culpable(tec_rios)) → SUCCESS: False

# backward_chain(kb, descartado(tec_rios)) → SUCCESS: True

# No hay error. No hay contradicción detectada. Solo prioridad de coartada.

```

¿Por qué el motor no detecta la contradicción?

El motor no detecta la contradicción porque las cláusulas de Horn son monotónicas: los hechos

nuevos solo pueden agregar información, no contradecir la existente. El predicado descartado(tec_rios) y el hecho acceso_registrado(tec_rios) coexisten en la KB sin generar ningún conflicto porque ninguna regla dice explícitamente que no pueden coexistir.

Para que el motor pueda razonar sobre inconsistencias, sería necesario añadir reglas explícitas de incompatibilidad, por ejemplo: contradiccion(\$X) :- descartado(\$X), evidencia_directa(\$X). Pero incluso así, el motor no lanzaría un error: simplemente derivaría el predicado contradiccion y el razonador de nivel superior tendría que interpretarlo.

Conclusión clave

El motor no detecta contradicciones lógicas: coexiste silenciosamente la evidencia con la coartada. El diseño de las reglas debe garantizar que sin_coartada sea un predicado mutuamente excluyente con coartada_verificada, lo cual es responsabilidad del modelador, no del motor de inferencia.

Pregunta 3: Forward chaining vs. Backward chaining en casos criminales

Pregunta

¿En qué se diferencia razonar con forward chaining vs. backward chaining al resolver un caso?
¿Cuándo conviene usar cada uno?

Diferencias estructurales

Dimensión	Forward Chaining	Backward Chaining
Dirección	Hechos → Conclusiones	Meta → Subobjetivos → Hechos
Punto de partida	Base de hechos conocidos	Una pregunta o meta específica
Objetivo	Derivar todo lo derivable	Probar o refutar una hipótesis
Eficiencia	Explora el espacio completo de	Explora solo el subgrafo necesario

	inferencias	
Uso típico	Análisis exploratorio de toda la KB	Verificar consultas específicas
Cuantificadores	No directamente orientado a $\exists/\forall$	Soporta ExistsGoal y ForallGoal

¿Cuándo conviene usar forward chaining?

El forward chaining es la herramienta adecuada cuando el objetivo es obtener una visión panorámica de todas las consecuencias que se derivan de la evidencia disponible. En el contexto de los casos criminales del taller, fue especialmente útil para:

- Derivar simultáneamente el estado de todos los sospechosos (culpable, descartado, sospechoso) en una sola pasada, sin necesidad de hacer una consulta separada por individuo.
- Detectar hechos derivados inesperados que emergen de la combinación de reglas, como coartadas cruzadas o redes activas, antes de formular preguntas específicas.
- En el caso La Red del Puerto de las Sombras, el forward chaining detecta automáticamente operacion_conjunta y red_activa sin que el detective tenga que hipotetizar explícitamente que existe una red.

¿Cuándo conviene usar backward chaining?

El backward chaining es el método correcto cuando ya se tiene una hipótesis específica que se quiere verificar o refutar. Es más eficiente porque no explora toda la KB: solo activa las reglas necesarias para probar o descartar la meta planteada. En el taller fue superior para:

- Responder preguntas concretas: ¿es Reynaldo culpable? ¿La acusación de la Marquesa es creíble? ¿Existe alguna red activa?
- Manejar cuantificadores (ExistsGoal, ForallGoal), que operan sobre metas estructuradas y necesitan el mecanismo de sustitución de variables que el backward chaining provee.
- Construir el árbol de prueba (proof tree) que permite al detective entender la cadena de razonamiento que llevó a la conclusión.

Conclusión clave

En la práctica, los dos métodos son complementarios: forward chaining al inicio de la investigación para explorar todos los hechos derivables, backward chaining para responder consultas específicas de forma eficiente y para manejar cuantificadores. El taller los combina en la TUI interactiva precisamente por esta razón.

Pregunta 4: Variables vs. constantes en la expresividad de las reglas

Pregunta

¿Qué rol juegan las variables vs. las constantes en la expresividad de las reglas? ¿Qué se perdería si solo se usaran constantes?

El rol de las variables

Las variables (prefijadas con \$ en el proyecto, como \$X, \$Y, \$A) permiten escribir reglas universalmente cuantificadas: una sola regla se aplica a cualquier individuo del dominio que satisfaga las condiciones del cuerpo. La unificación es el mecanismo que liga una variable a un valor concreto durante la inferencia.

```
# Una sola regla cubre todos los sospechosos del dominio

kb.add_rule(Rule(

    head=Predicate('culpable', (Term('$X'),)),

    body=(

        Predicate('evidencia_directa', (Term('$X'),)),

        Predicate('sin_coartada', (Term('$X'),)),

    )

))

# Esta regla se instancia para reynaldo, elena, tec_rios, sobrino_esteban,

# oficial_duarte, marinero_pinto... cualquier constante que satisfaga el cuerpo.
```

Las variables también permiten modelar relaciones entre individuos distintos. La regla de coartada cruzada, por ejemplo, necesita dos variables diferentes para capturar que dos personas distintas se dan coartada mutuamente:

```
# Regla con dos variables que se relacionan entre sí

kb.add_rule(Rule(

    head=Predicate('coartada_cruzada', (Term('$X'), Term('$Y'))),

    body=(

        Predicate('da_coartada_a', (Term('$X'), Term('$Y'))),

        Predicate('da_coartada_a', (Term('$Y'), Term('$X'))),

    )

))
```

¿Qué se perdería si solo se usarán constantes?

Si las reglas solo usaran constantes, el sistema degeneraría en un sistema de reglas específicas por individuo. Esto implica pérdidas concretas en tres dimensiones:

1. Escalabilidad: Para un caso con N sospechosos, se necesitarían N copias de cada regla general (una para reynaldo, una para pablo, una para bernardo...). En el caso más complejo del taller (La Red del Puerto) con 4 sospechosos y 10 reglas, esto significaría 40 reglas en lugar de 10.
2. Generalización: No habría forma de expresar la idea de que cualquier sospechoso que cumpla ciertas condiciones es culpable. Se perdería la separación entre la lógica del razonamiento y los datos del caso, que es precisamente lo que hace potente al enfoque de la lógica de predicados.
3. Cuantificadores: Los cuantificadores ExistsGoal y ForallGoal perderían todo sentido sin variables. ¿Existe algún X culpable? es imposible de formular si todas las reglas son sobre individuos concretos. La pregunta siempre se tendría que convertir en ¿es reynaldo culpable? o ¿es pablo culpable?, sin la capacidad de generalizar.

Conclusión clave

Las variables son lo que hace a la lógica de predicados de primer orden más expresiva que la lógica proposicional. Sin variables, el sistema equivale en poder expresivo a un conjunto de tablas de lookup: no hay inferencia general, solo recuperación de hechos específicos.

Pregunta 5: La suposición de mundo cerrado (CWA)

Pregunta

¿Cómo afecta la suposición de mundo cerrado (lo que no se puede probar es falso) a las conclusiones sobre sospechosos que no tienen evidencia en su contra ni coartada verificada?

La suposición de mundo cerrado (CWA)

La suposición de mundo cerrado establece que todo lo que no puede ser derivado a partir de los hechos y reglas de la KB se considera falso. En un sistema de cláusulas de Horn como el implementado en el taller, esto significa que predicados como `sin_coartada`, `motivo_economico` o `lejos_de_escena` son falsos para cualquier individuo para el que no existan hechos o reglas que los generen.

Impacto en sospechosos sin información

Consideremos un sospechoso hipotético llamado `desconocido_x` que aparece mencionado en la narrativa pero para el que no se han añadido hechos explícitos de ningún tipo. Bajo CWA:

- `sin_coartada(desconocido_x)` es FALSO (porque no hay ningún hecho ni regla que lo derive).
- `evidencia_directa(desconocido_x)` es FALSO (misma razón).
- Por tanto, `culpable(desconocido_x)` es FALSO (sus premisas son falsas).
- `descartado(desconocido_x)` también es FALSO (no tiene coartada verificada derivada).

El resultado es que el sospechoso queda en un estado de total ambigüedad: no culpable, no descartado. El motor simplemente lo ignora. Esta es una limitación importante del modelo.

Comparación con el mundo real

En una investigación real, un sospechoso sin evidencia en ninguna dirección no es equivalente a alguien inocente ni a alguien culpable. Es simplemente alguien sobre quien no se tiene

información suficiente. Sin embargo, bajo CWA, el sistema lo trata como si todas sus propiedades fueran falsas, lo que podría llevar a conclusiones incorrectas si se añaden reglas que razonan sobre la ausencia de evidencia.

Un ejemplo concreto del taller: el Asistente Mora en el caso Pharmax tiene acceso registrado a la sala y no tiene coartada, pero no tiene motivo económico derivable. Bajo CWA, `motivo_economico(asistente_mora)` es falso, por lo que la regla de culpabilidad no se dispara para él. Si el diseñador del caso omitiera accidentalmente el hecho que genera el motivo de Ríos, el sistema tampoco lo marcaría culpable, fallando silenciosamente.

Conclusión clave

La CWA convierte la ausencia de información en negación activa: un sospechoso sin datos explícitos no es 'desconocido' sino 'sin propiedades relevantes'. Esto hace crítico que el modelador sea exhaustivo al definir los hechos. Una omisión no genera un error; genera una conclusión incorrecta que el sistema acepta como válida.

Punto 4: Reflexión (10%)

4a. Reflexión sobre la implementación de cada punto (5%)

A continuación se describen el enfoque, las dificultades encontradas y los métodos de verificación utilizados para cada uno de los tres puntos de implementación del taller.

Punto 1 — Model Checking Proposicional

Enfoque de implementación: la función central `get_all_models` fue abordada mediante representación binaria. Cada entero en el rango $[0, 2^n - 1]$ codifica una asignación de verdad completa: el bit j -ésimo (del más significativo al menos significativo) determina el valor booleano del j -ésimo átomo en orden lexicográfico. Esta representación garantiza la generación exhaustiva y no duplicada de los 2^n modelos.

Para `check_entailment` se aplicó el teorema de refutación: $KB \models q$ si y solo si $KB \wedge \neg q$ es insatisfacible. Esto evita tener que iterar sobre modelos manteniendo la condición de que la KB sea verdadera, simplificando el código considerablemente.

Dificultades encontradas: el mayor desafío fue el ordenamiento de los átomos. Al usar un set

de Python, el orden de iteración no es determinista entre ejecuciones. Fue necesario llamar explícitamente a `sorted(atoms)` para garantizar que el bit más significativo siempre corresponda al primer átomo en orden lexicográfico, produciendo tablas de verdad reproducibles.

Verificación: se ejecutó la batería completa de `tests/test_model_checking.py` (31 casos). Se verificaron manualmente los resultados del caso criminal proporcionado en el notebook `parte1_model_checking.ipynb`: la KB con `jardín_mojado` deriva correctamente que llovía. Se añadieron casos de frontera propios: un conjunto vacío de átomos (debe producir un solo modelo vacío) y fórmulas con un solo átomo.

Punto 2 — Transformación a CNF

Enfoque de implementación: todas las funciones siguen el mismo patrón de recursión sobre el AST (árbol sintáctico abstracto). Cada función inspecciona el tipo del nodo raíz con `isinstance` y, dependiendo del tipo, o bien aplica la transformación correspondiente o bien desciende recursivamente a los nodos hijos, reconstruyendo el árbol con los resultados.

La función más compleja fue `distribute_or_over_and`. Para un nodo `Or` con múltiples hijos donde más de uno puede ser un `And` (caso `Or(And(a,b), c, And(d,e))`), se adoptó una estrategia de detección del primer hijo `And` encontrado, seguida de la generación de la conjunción distribuida, y luego recursión sobre el resultado para manejar distribuciones anidadas.

Dificultades encontradas: el caso `Or(And(a,b), And(c,d))` fue inicialmente mal manejado porque la implementación solo detectaba el primer `And`. La solución fue garantizar que `distribute_or_over_and` se llame recursivamente sobre el resultado de cada distribución, de modo que los `And` restantes en la lista de `others` sean eventualmente procesados. La prueba `test_or_with_multiple_and_children` fue clave para detectar este bug.

Verificación: se ejecutó `tests/test_cnf.py` completo (23 casos). Se verificó la equivalencia lógica de la fórmula original con su CNF usando la función auxiliar `_is_equivalent` del archivo de `tests`, que compara los resultados en todos los modelos. Se construyeron manualmente las CNF esperadas para fórmulas simples ($p \rightarrow q \rightarrow \neg p \vee q$) y se compararon con la salida de `to_cnf`.

Punto 3 — Lógica de Predicados: Casos Criminales

Enfoque de implementación: se siguió un proceso sistemático para cada caso: (1) leer el docstring completo y separar mentalmente los hechos (primer párrafo) de las reglas de deducción (segundo párrafo); (2) mapear cada línea del docstring a un `add_fact` o `add_rule` con la estructura de predicados y variables correcta; (3) verificar que las constantes usadas en los hechos coincidan con los términos que aparecen en las consultas del CASE.

La convención de variables con prefijo `$` fue crítica para distinguirlas de las constantes. Se adoptó el hábito de nombrar las variables con nombres descriptivos (`$X` para persona, `$A` para arma/objeto) para mejorar la legibilidad de las reglas complejas.

Dificultades encontradas: el caso La Red del Puerto de las Sombras presentó la mayor dificultad porque involucra dos culpables y relaciones entre ellos (`operacion_conjunta`,

red_activa), lo que requirió razonar cuidadosamente sobre qué predicado actúa como dominio en el ForallGoal y cuál como propiedad. También fue desafiante modelar que una red está activa si al menos uno de sus miembros es culpable, ya que requiere introducir el cartel como constante en la KB.

Verificación: se ejecutó tests/test_predicates.py completo. Para cada caso, se verificó manualmente que backward_chain retornara success=True para todas las consultas del CASE. Se usó también la TUI interactiva (uv run main.py) para ejecutar el modo backward wizard y confirmar visualmente el árbol de prueba de cada consulta crítica.

4b. Reflexión sobre la co-construcción con IA (5%)

Contexto de uso de IA en el proyecto

El equipo desarrolló una primera versión autónoma de cada función antes de consultar herramientas de IA. La IA se utilizó en tres momentos específicos: depuración de bugs puntuales, refactorización del código para mejorar su legibilidad, y apoyo en la formulación de las respuestas analíticas. A continuación se describen los aprendizajes, la naturaleza de las correcciones y los límites observados.

Aprendizajes obtenidos

El principal aprendizaje del proceso de co-construcción fue que la IA resulta más valiosa como herramienta de depuración y explicación que como generadora de código desde cero. Cuando se le presentó una implementación propia y un test fallido, la IA fue capaz de identificar el error y explicar por qué ocurría con un nivel de detalle que habría tomado horas de debugging manual.

Un segundo aprendizaje fue sobre los límites del prompt. Las primeras consultas sobre distribute_or_over_and con prompts vagos ('mi función falla, ¿qué está mal?') produjeron respuestas genéricas poco útiles. Al reformular el prompt incluyendo el código propio, el test que fallaba y el resultado esperado vs. obtenido, la calidad de la respuesta mejoró drásticamente. Este proceso enseñó que la precisión del prompt es una habilidad en sí misma.

Un tercer aprendizaje fue conceptual: al pedir a la IA que explicara la diferencia entre el comportamiento de CWA en motores de Horn versus en bases de datos deductivas con negación como fallo (NAF), la explicación generada enriqueció la comprensión más allá de lo que se habría obtenido leyendo el código solo.

Naturaleza de las correcciones de la IA

Las correcciones fueron predominantemente secundarias o de refinamiento, no correcciones fundamentales. Las versiones iniciales del equipo tenían la lógica correcta en el 80% de los casos; los errores eran de implementación (orden de bits incorrecto en `get_all_models`, manejo del caso base de `flatten` cuando queda un solo elemento) más que de comprensión conceptual.

La única corrección que podría considerarse sustancial fue en `distribute_or_over_and`: el equipo no había contemplado el caso de Or con múltiples hijos And. La IA identificó que la recursión debía aplicarse sobre el resultado completo, no solo sobre el primer And encontrado. Esto sí fue un cambio en la lógica del algoritmo, aunque la estructura general de la función ya era correcta.

En el modelado de casos criminales (Punto 3), la IA no fue utilizada para generar las KB directamente, sino para verificar la coherencia de las reglas escritas por el equipo ante los queries esperados. Este uso como 'revisor lógico' fue el más valioso del proyecto.

Beneficios y limitaciones de la IA en este contexto

Los beneficios más claros fueron la velocidad de depuración (bugs que habrían tomado 1-2 horas de debugging se identificaron en minutos), la capacidad de generar ejemplos contra-intuitivos para verificar casos de frontera, y la mejora en la calidad explicativa de las respuestas analíticas del Punto 3b.

Las limitaciones también fueron evidentes. La IA ocasionalmente generó código que parecía correcto pero fallaba en casos de frontera (como la gestión de sets vacíos en `get_all_models` o la recursión infinita en `backward_chain` con ciclos). Fue necesario verificar siempre la salida con los tests antes de confiar en ella. Adicionalmente, la IA no tiene acceso al contexto específico del proyecto (la estructura de las clases, las convenciones de variables con \$), por lo que las sugerencias de código siempre requerían adaptación antes de ser aplicadas.

La conclusión del equipo es que la IA en este tipo de taller funciona mejor como un tutor bajo demanda que como un desarrollador autónomo. El valor máximo se obtiene cuando el estudiante ya comprende el problema y usa la IA para validar, refinar y profundizar, no para obtener una solución de partida.